

Today 8:56 PM

45_snake.zip
File

You are my pair-programming partner for a Python + Pygame Snake game. I will give you 4 tasks one at a time; this is Task 1. I'm attaching my project as a zip file (45_snake.zip) containing main.py, requirements.txt, README.md and the game/ folder (game_engine.py, snake.py, food.py). Unzip it and read all the files before making changes.

How the game currently works:

- 600x600 window, 20px cells -> 30x30 grid, runs at 60 FPS.
- The snake moves one cell every (60 // moves_per_second) frames.
- snake.body is a list of (x, y) grid cells, head = body[0].
- Key presses call engine.handle_keydown(), which calls snake.set_direction() immediately.

THE BUG:

If the snake is moving Right and I press Left, it dies instantly because the head moves straight into the segment behind it. set_direction() has no guard against reversing. There is a second hidden version of this bug: because key presses are applied instantly but the snake only moves every few frames, if I'm moving Right and quickly press Up then Left before the next move, the direction ends up as Left and the snake still reverses into itself.

WHAT I NEED YOU TO FIX:

1. Block reversing: the snake must never turn 180 degrees into its own neck.
2. The check must be against the direction of the LAST ACTUAL MOVE (or the last queued turn), NOT just the last key pressed, so the quick double-key-press case is also fixed.
3. Add a small turn queue (max 2 turns) in snake.py so fast inputs like Up then Left are applied on consecutive moves instead of being lost. Ignore a key if it's the same as the last queued direction or its

exact opposite.

4. In `move()`, take the next turn from the queue (if any) before moving.
5. Keep wall collision accurate: the snake dies only when the head goes outside the 30x30 grid.
6. Keep self collision accurate: moving into the cell the tail is leaving on this same step is allowed (the tail is removed before the self-collision check), unless the snake is growing.
7. Food must never spawn on the snake, INCLUDING the very first food (right now `Food.__init__` calls `respawn([])` with an empty list, so the first food can land on the snake). Pass the snake's body into `Food` when it's created. Pick a random cell from the list of free cells instead of retrying in a loop, and handle the board being full without crashing.
8. Check food eating by comparing the head's grid cell with the food's grid cell, instead of using rect collision.

RULES:

- Don't add any other features yet (no game over screen, no menus, no sound). Those are later tasks.
- Keep the existing structure and class/method names so the other files still work.
- When you're done, give me the SAME zip file back (45_snake.zip) with the updated files inside, keeping the exact same folder structure (`main.py`, `requirements.txt`, `README.md`, `game/game_engine.py`, `game/snake.py`, `game/food.py`). Don't rename, add or remove any files.
- Also show the COMPLETE updated code for every file you changed in the chat, clearly labelled with its file path, so it's visible in my chat history.
- After the code, explain in 3-5 bullet points what you changed and why, and tell me exactly how to test that the bug is fixed.

Analysis errored

Worked for 55s

Done. I unzipped and read the project files first, then updated only the files needed for Task 1.

Download

 Download the updated 45_snake.zip

Complete updated code

45_snake/game/snake.py

 Python

```
import pygame

class Snake:
    def __init__(self, x, y, cell_size):
        self.cell_size = cell_size
        # body is a list of (x, y) grid-cell positions, head is body[0]
        self.body = [(x, y), (x - 1, y), (x - 2, y)]
        self.direction = (1, 0) # direction of the last actual move
        self.turn_queue = []
        self.max_turn_queue = 2
        self.grow_pending = False

    def set_direction(self, dx, dy):
        new_direction = (dx, dy)

        # Compare against the last actual move when the queue is empty,
        # otherwise compare against the last queued turn. This prevents
        # rapid key presses from eventually creating a 180-degree turn.
        reference_direction = self.turn_queue[-1] if self.turn_queue else self.direction
```

45_snake/game/food.py

 Python

```
import pygame
import random

class Food:
    def __init__(self, grid_width, grid_height, cell_size, occupied_cells=None):
        self.grid_width = grid_width
        self.grid_height = grid_height
        self.cell_size = cell_size
        self.x = None
        self.y = None
        self.respawn(occupied_cells if occupied_cells is not None else [])
```

```
def respawn(self, occupied_cells):
    occupied = set(occupied_cells)
    free_cells = [
        (x, y)
        for y in range(self.grid_height)
        for x in range(self.grid_width)
        if (x, y) not in occupied
```

45_snake/game/game_engine.py

</> Python

```
import pygame
from .snake import Snake
from .food import Food

# Game Engine

WHITE = (255, 255, 255)
GREEN = (0, 200, 0)
RED = (220, 60, 60)

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.cell_size = 20
        self.grid_width = width // self.cell_size
        self.grid_height = height // self.cell_size

        self.snake = Snake(
            self.grid_width // 2,
```

main.py , requirements.txt , and README.md were not changed.

What changed

- **180° reversals are blocked** using the direction of the actual move, or the most recent queued turn—not simply the most recent key press.
- Added a **maximum 2-turn queue**, so Up → Left during one movement interval becomes two consecutive turns instead of causing an instant reversal.
- move() now consumes the next queued turn **before calculating the new head position**.
- Self-collision remains grid-accurate: the **tail is removed before collision checking** unless the snake is growing. Wall collision still means simply going outside 0..29 .

- Food now receives `snake.body` during construction, chooses randomly from **free cells**, safely handles a completely full board, and eating is checked using **grid coordinates rather than rectangle collision**.

I also ran syntax/logic tests covering direct reversal, rapid `Up → Left`, duplicate/opposite queued turns, tail-cell movement, wall boundaries, initial food placement, and a full-board food case. The tests passed.

Exactly how to test the bug

1. Run:

 **Bash**

```
pip install -r requirements.txt
python main.py
```

2. Direct reversal test

- Snake starts moving right.
- Immediately press **Left**.
- It should **continue right** rather than instantly dying.

3. Hidden rapid-input test

- While moving right, quickly press:
Up → Left
- Before the next snake movement happens, both keys may be registered.
- The snake should move **Up on the next step, then Left on the following step**.
- It must **not** reverse directly into its neck.

4. Queue test

- While moving in one direction, rapidly enter two valid turns such as:
Up → Left
- Confirm both turns happen across consecutive grid movements.

5. Food test

- Restart several times.
- The initial food should never appear underneath any of the three starting snake segments.

6. Wall test

- Drive directly into a wall.
- The snake should die only when its head moves outside the 30×30 grid.

7. Self-collision test

- Make the snake grow and create a loop.
- Moving into the **tail's current cell on the same step that the tail leaves it** should remain valid; moving into any segment that remains should cause collision.

</>

Task 1 works perfectly. Now Task 2: Implement a proper Game Over screen.

CURRENT PROBLEM:

When the snake hits a wall or itself, `update()` sets `self.game_over = True` and `render()` just prints "Game over! Final score: X" to the console once. The game window freezes on the last frame with no message, and the player has no way to know the game ended or what to do next.

WHAT I NEED:

1. Remove the console print and the `_game_over_logged` flag completely.
2. When `game_over` is `True`, `render()` should still draw the snake, food and score, then draw a semi-transparent dark overlay over the whole 600x600 window (use a `pygame.Surface` with `SRCALPHA`, black with alpha around 180).
3. On top of the overlay, draw centered text:
 - "GAME OVER" in a large bold red font (around size 64), slightly above the center.
 - "Final Score: X" in white (around size 30) just below it.
 - "Press Esc or Q to quit" in gray (around size 24) below that.Write a small helper method like `_blit_centered(screen, text, font, color, y)` to center text horizontally, and create the fonts once in `__init__` (not every frame).
4. The game must WAIT for input on this screen instead of closing or freezing:
 - While `game_over` is `True`, arrow/WASD keys should be ignored (the snake must not change direction).
 - Pressing Esc or Q should quit cleanly: add a `self.quit_requested` flag in `GameEngine`, set it to `True` in `handle_keydown`, and in `main.py` exit the while loop when `engine.quit_requested` is `True` so `pygame.quit()` runs normally.
 - Closing the window with the X button must still work.
5. Move the "game over" logic into a small `end_game()` method in `GameEngine` that sets `game_over = True`, and call it from both the

wall and self-collision checks in update(). (I'll hook more things into it in later tasks.)

6. Put the snake/food/score setup from `__init__` into a `reset()` method and call it from `__init__`, so the game can be restarted later. Don't add a restart key yet, that's Task 3.

RULES:

- Keep ALL the Task 1 fixes exactly as they are (turn queue, reversal blocking, food spawning on free cells, grid-based food eating).
- Don't add difficulty selection, replay or sound yet. Those are Tasks 3 and 4.
- Give me the SAME zip file back (45_snake.zip) with the updated files inside, keeping the exact same folder structure, and don't include `__pycache__` folders.
- Also show the COMPLETE updated code for every file you changed in the chat, clearly labelled with its file path.
- After the code, explain in 3-5 bullet points what you changed and why, and tell me exactly how to test it.

Worked for 28s

Task 2 is implemented, while keeping **all Task 1 fixes unchanged**.

 [Download the updated 45_snake.zip](#)

Complete updated code

45_snake/main.py

 Python

```
import pygame
from game.game_engine import GameEngine

# Initialize pygame/Start application
pygame.init()

# Screen dimensions
WIDTH, HEIGHT = 600, 600
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Snake - Pygame Version")
```

```
# Colors
WHITE = (255, 255, 255)
BLACK = (0, 0, 0)

# Clock
clock = pygame.time.Clock()
FPS = 60

# Game loop
```

45_snake/game/game_engine.py

</> Python

```
import pygame
from .snake import Snake
from .food import Food

# Game Engine

WHITE = (255, 255, 255)
GREEN = (0, 200, 0)
RED = (220, 60, 60)
GRAY = (180, 180, 180)
BLACK = (0, 0, 0)

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.cell_size = 20
        self.grid_width = width // self.cell_size
        self.grid_height = height // self.cell_size
```

45_snake/game/snake.py

Unchanged from Task 1, so its complete code remains:

</> Python

```
import pygame

class Snake:
    def __init__(self, x, y, cell_size):
```

```

        self.cell_size = cell_size
        # body is a list of (x, y) grid-cell positions, head is body[0]
        self.body = [(x, y), (x - 1, y), (x - 2, y)]
        self.direction = (1, 0) # direction of the last actual move
        self.turn_queue = []
        self.max_turn_queue = 2
        self.grow_pending = False

def set_direction(self, dx, dy):
    new_direction = (dx, dy)

    # Compare against the last actual move when the queue is empty,
    # otherwise compare against the last queued turn. This prevents
    # rapid key presses from eventually creating a 180-degree turn.
    reference_direction = self.turn_queue[-1] if self.turn_queue else self.direction

```

45_snake/game/food.py

Unchanged from Task 1:

</> Python

```

import pygame
import random

class Food:
    def __init__(self, grid_width, grid_height, cell_size, occupied_cells=None):
        self.grid_width = grid_width
        self.grid_height = grid_height
        self.cell_size = cell_size
        self.x = None
        self.y = None
        self.respawn(occupied_cells if occupied_cells is not None else [])

    def respawn(self, occupied_cells):
        occupied = set(occupied_cells)
        free_cells = [
            (x, y)
            for y in range(self.grid_height)
            for x in range(self.grid_width)
            if (x, y) not in occupied
        ]

```

requirements.txt and README.md were not changed.

What changed

- Added a proper **Game Over overlay** with `GAME OVER`, final score, and quit instructions. The snake, food, and score remain visible underneath.
- Added `end_game()` and routed both wall and self-collision through it.
- Removed `_game_over_logged` and the console print entirely.
- Added `quit_requested`; **Esc/Q quit only while on the Game Over screen**, while movement keys are ignored there.
- Moved game-state creation into `reset()`, preparing the engine for the restart functionality in Task 3.

Exactly how to test

1. Run:

```
</> Bash
```

```
python main.py
```

2. Wall Game Over

- Keep moving right until the snake hits the right wall.
- The window should remain open.
- You should see:
 - `GAME OVER`
 - `Final Score: X`
 - `Press Esc or Q to quit`
- The screen should have a dark translucent overlay.

3. Self-collision Game Over

- Grow the snake using food.
- Make it turn around into its body.
- The same Game Over screen should appear.

4. Input lock

- Once Game Over appears, press arrow keys and WASD.
- The snake must **not move or change direction**.

5. Quit

- Press `Esc` → window should close cleanly.
- Run it again and press `Q` on Game Over → same result.
- Click the window's **X** → it should still close normally.

6. Task 1 regression

- Before dying, test `Up` → `Left` rapidly while moving right.

- The turn queue/reversal protection should still behave exactly as in Task 1.

The ZIP contains exactly the original required files and **no __pycache__ folders**.

Nice, the game over screen works great! Okay, Task 3 now. I want to add a replay option.

Right now once you die, the only thing you can do is press Esc/Q to quit, which is kinda annoying. I want the player to be able to play again straight from the Game Over screen and pick how fast the snake goes.

Here's what I'm thinking:

- Three difficulty levels: Easy (6 moves per second), Medium (10) and Hard (15). Let the player pick with 1/2/3, or E/M/H, and the numpad keys should work too. The first game when you open it can just start on Medium.
 - When you pick one on the Game Over screen, it should start a fresh game at that speed. Snake back in the middle, new food (still not on the snake), score back to 0. You already made a `reset()` method in Task 2, so just reuse that.
 - Esc/Q should still quit like before, and arrow keys/WASD should still do nothing on the Game Over screen.
 - Change the Game Over text a bit. Keep "GAME OVER" and the final score, but replace "Press Esc or Q to quit" with:
 - "Play again - choose difficulty:" (white)
 - "1 Easy 2 Medium 3 Hard" (green, slightly bigger)
 - "Esc / Q to quit" (gray)
- Just make sure everything's spaced nicely and nothing overlaps.
- While playing, could you show the current difficulty (like "Medium") in small gray text in the top-right corner? The score is already on the top-left, so it won't clash.

Please don't break anything from Tasks 1 and 2. The no-reversing fix, the turn queue, food spawning and the game over screen should all keep working the same. And no sound yet, I'll ask for that next. Also create the fonts once in `__init__`, not every frame.

Same as before: send me back the same 45_snake.zip with the updated files (same folders, no __pycache__), paste the full code of any file you changed here in the chat, and give me a quick explanation of what you changed plus how to test it.

Worked for 23s

Task 3 is done. I kept the Task 1 and Task 2 behavior intact, including the turn queue, reversal protection, safe food spawning, grid-based eating, and Game Over overlay.

 [Download the updated 45_snake.zip](#)

Changed file: 45_snake/game/game_engine.py

 Python

```
import pygame
from .snake import Snake
from .food import Food

# Game Engine

WHITE = (255, 255, 255)
GREEN = (0, 200, 0)
RED = (220, 60, 60)
GRAY = (180, 180, 180)
BLACK = (0, 0, 0)

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.cell_size = 20
        self.grid_width = width // self.cell_size
        self.grid_height = height // self.cell_size
```

main.py , snake.py , food.py , requirements.txt , and README.md are unchanged from Task 2.

What changed

- Added **Easy = 6, Medium = 10, Hard = 15 moves/sec**. New games default to **Medium**.
- On Game Over, 1/2/3 , numpad 1/2/3 , or E/M/H calls `start_new_game()` , updates the speed, and reuses `reset()` .

- The Game Over screen now has the requested difficulty-selection text and spacing.
- Added the current difficulty to the **top-right HUD** in small gray text.
- All fonts are created once in `__init__` ; no font creation happens during rendering.

How to test

1. Start the game:

 **Bash**

```
python main.py
```

Confirm **Medium** appears in the top-right.

2. Die by hitting a wall.
3. On Game Over, test:
 - 1 or E → new **Easy** game.
 - 2 or M → new **Medium** game.
 - 3 or H → new **Hard** game.
 - Numpad 1/2/3 → same respective difficulties.
4. After each replay, verify:
 - Snake starts back in the center.
 - Score is 0 .
 - Food is not underneath the snake.
 - Difficulty label changes correctly.
 - Snake speed actually changes.
5. On Game Over, press arrow/WASD keys. **Nothing should happen.**
6. Press `Esc` or `Q` . The game should close normally.
7. Finally, test the Task 1 `Up` → `Left` rapid-input case again to make sure the replay changes haven't affected the turn queue/reversal protection.

Awesome, replay works really well! Last one, Task 4: I want to add some sound effects.

Nothing fancy, just two sounds:

- A short, happy little beep when the snake eats food (like two quick rising notes).
- A sadder falling tone when the game ends (a few notes going down, maybe half a second long).

A couple of things I'd like:

- I don't want to download or add any .wav/.mp3 files, so could you generate the sounds in code? Just use pygame.mixer with Python's built-in math and struct modules, and `pygame.mixer.Sound(buffer=...)`. Please no numpy, I don't want extra installs.
- Add a little fade-out on each note so it doesn't click or pop.
- Put the sound stuff in its own file, like `game/sounds.py` with a small `Sounds` class (`play_eat()` and `play_game_over()`), so the engine stays clean.
- Play the eat sound at the point where the score goes up, and the game over sound inside `end_game()` so it plays for both wall and self crashes. It should only play once per death, not every frame.
- Really important: if someone's PC has no audio device or the mixer fails to start, the game should still run normally, just silently. No crashes.
- Make sure the mixer is set up to match the sound format (e.g. call `pygame.mixer.pre_init` before `pygame.init()` in `main.py`), so the sounds don't come out at the wrong pitch.

Please keep everything from Tasks 1-3 working exactly the same: no-reversing, the turn queue, food spawning, the game over screen, replay and difficulty.

Same as before: send me back the same `45_snake.zip` with the updated files plus the new `sounds.py` (same folders, no `__pycache__`), paste the full code of every file you changed or added here in the chat, and give me a quick explanation of what you changed plus how to test it.

Worked for 24s

Task 4 is complete. I kept Tasks 1–3 intact and added only the requested sound functionality.

 [Download the updated 45_snake.zip](#)

Changed files

`45_snake/main.py`

</> Python

```
import pygame

# Match pygame mixer output to the generated 16-bit mono sounds.
pygame.mixer.pre_init(frequency=44100, size=-16, channels=1, buffer=512)

from game.game_engine import GameEngine

# Initialize pygame/Start application
pygame.init()

# Screen dimensions
WIDTH, HEIGHT = 600, 600
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Snake - Pygame Version")

# Colors
WHITE = (255, 255, 255)
BLACK = (0, 0, 0)

# Clock
clock = pygame.time.Clock()
```

45_snake/game/game_engine.py

</> Python

```
import pygame
from .snake import Snake
from .food import Food
from .sounds import Sounds

# Game Engine

WHITE = (255, 255, 255)
GREEN = (0, 200, 0)
RED = (220, 60, 60)
GRAY = (180, 180, 180)
BLACK = (0, 0, 0)

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.cell_size = 20
```

```
self.grid_width = width // self.cell_size
```

45_snake/game/sounds.py — new file

</> Python

```
import math
import struct

import pygame

class Sounds:
    def __init__(self):
        self.enabled = False
        self.sample_rate = 44100
        self.eat_sound = None
        self.game_over_sound = None

    try:
        if not pygame.mixer.get_init():
            pygame.mixer.init(
                frequency=self.sample_rate,
                size=-16,
                channels=1,
                buffer=512
            )
```

snake.py , food.py , requirements.txt , and README.md remain unchanged from Task 3.

What changed

- Added game/sounds.py with a small Sounds class. The tones are generated entirely with **math** + **struct** , then loaded through `pygame.mixer.Sound(buffer=...)` — no audio files and no NumPy.
- The eat sound is two short **rising notes**, while Game Over is three **falling notes**. Each individual note has a short fade-in/out to prevent clicks.
- `play_eat()` happens exactly when the score increases. `play_game_over()` happens inside `end_game()` , which now also guards against being called twice, so the death sound plays only once.
- `pygame.mixer.pre_init()` is called **before** `pygame.init()` with the same 44.1 kHz, 16-bit mono format used for generated audio.
- Audio initialization/playback is wrapped so a missing/failed audio device simply means **silent gameplay** rather than a crash.

How to test

1. Run normally:

 **Bash**

```
python main.py
```

2. Eat a food item.

- You should hear a short **two-note rising beep**.
- Score should still increase normally.

3. Die by hitting a wall.

- You should hear the **falling Game Over tone once**.
- The existing Game Over screen should appear exactly as before.

4. Die by hitting yourself.

- Again, the falling tone should play once.

5. Replay from Game Over with 1/2/3 or E/M/H .

- The new game should work normally.
- Eat food and verify the eat sound still works.
- Die again and verify the Game Over sound plays again for the new death.

6. Regression-test Tasks 1–3:

- Up → Left rapid input still cannot cause a 180° reversal.
- Food never spawns inside the snake.
- Game Over overlay still works.
- Difficulty selection still works.
- Esc/Q still quits.
- Arrow/WASD still do nothing on the Game Over screen.

7. **Audio-failure behavior:** if the machine cannot initialize the mixer/audio device, the game should continue running normally without throwing an exception.